

MLGWorks Utility Framework

Tim Schlachter

July 21, 2025

Contents

1	Timer Utility	3
1.1	Overview	3
1.2	Construction	3
1.3	Core Methods	3
1.4	Properties	3
1.5	Events	3
1.6	Typical Usage Example	4
1.7	Use Cases	4
2	Event Bus	5
2.1	Overview	5
2.2	Design	5
2.3	Interface: <code>IEvent</code>	5
2.4	Subscribing to Events	5
2.5	Unsubscribing from Events	5
2.6	Publishing Events	5
2.7	Implementation Summary	6
2.8	Example Event Type	6
2.9	Threading Note	6
2.10	Use Cases	6
3	Generic Singleton Base Class	7
3.1	Overview	7
3.2	Purpose	7
3.3	Usage	7
3.4	Key Features	7
3.5	Thread Safety	8
3.6	Error Handling	8
3.7	Limitations	8
4	State Machine	9
4.1	Overview	9
4.2	Interfaces	9
4.3	Usage	9
4.4	Features	10
4.5	Best Practices	10
5	Dependency Injection System	11
5.1	Overview	11

5.2	Registering Services	11
5.3	Dependency Injection	11
5.4	Lifecycle Initialization	12
5.5	Execution Order	12
5.6	Best Practices	12
6	Logger Utility	13
6.1	Overview	13
6.2	LogLevel and Targets	13
6.3	Configuration Options	13
6.4	Logging API	14
6.5	Unity Integration	14
6.6	Lifecycle	14
6.7	Log Cleanup	14
6.8	Example Usage	14
6.9	Testing	14
6.10	Editor Support	14
6.11	Implementation Notes	15

1 Timer Utility

1.1 Overview

The `MLGWorks.Utils.Helpers.Timer` class is a lightweight, pure C# utility for managing countdown timers with support for starting, pausing, resuming, and resetting. This is useful for implementing custom timing logic in non-MonoBehaviour systems.

1.2 Construction

To create a new timer, instantiate it with a duration in seconds:

```
Timer timer = new Timer(5f); // 5 second timer
```

1.3 Core Methods

`Start()` Starts or restarts the timer from zero.

`Reset()` Stops and resets the timer to zero.

`Pause()` Temporarily stops the timer's progress without resetting.

`Resume()` Continues a paused timer.

`Update(deltaTime)` Advances the timer's internal clock by `deltaTime`. This should be called every frame:

```
timer.Update(Time.deltaTime);
```

1.4 Properties

- `Duration` – The total time (in seconds) the timer is set for.
- `Elapsed` – The current elapsed time.
- `IsRunning` – True if the timer is active and not finished.
- `IsPaused` – True if the timer is currently paused.
- `IsFinished` – True if the timer has completed.

1.5 Events

`OnFinished` Event that is fired exactly once when the timer reaches its duration. Useful for triggering delayed actions:

```
timer.OnFinished += () => Debug.Log("Timer complete!");
```

1.6 Typical Usage Example

```
public class TimerExample : MonoBehaviour
{
    private Timer myTimer;

    void Start()
    {
        myTimer = new Timer(3f);
        myTimer.OnFinished += OnTimerDone;
        myTimer.Start();
    }

    void Update()
    {
        myTimer.Update(Time.deltaTime);
    }

    void OnTimerDone()
    {
        Debug.Log("Timer finished!");
    }
}
```

1.7 Use Cases

- Cooldown mechanics
- Timeouts or delays
- Non-Monobehaviour timing in services or logic layers
- Flexible, testable alternative to `Coroutine` or `Invoke`

2 Event Bus

2.1 Overview

The `EventBus` in `MLGWorks.Utils.Patterns` provides a static publish-subscribe (pub-sub) messaging system for decoupled communication across game systems. Components can subscribe to specific event types and respond when those events are published.

2.2 Design

- Events must implement the `IEvent` interface.
- Subscriptions are strongly typed and internally wrapped into generic `Action<IEvent>` delegates.
- Subscriptions are stored in a `Dictionary<Type, List<Action<IEvent>>>`.
- A second dictionary maps original delegates to wrapped delegates to support clean unsubscription.

2.3 Interface: IEvent

Events must implement:

```
public interface IEvent
{
    string Name { get; }
}
```

This allows each event to optionally include a name identifier (for logging or debugging).

2.4 Subscribing to Events

Use the `Subscribe<T>` method to listen for a specific event type:

```
EventBus.Subscribe<PlayerDiedEvent>(OnPlayerDied);
```

Where:

```
void OnPlayerDied(PlayerDiedEvent e)
{
    Debug.Log($"{e.PlayerName} has died.");
}
```

2.5 Unsubscribing from Events

Use the `Unsubscribe<T>` method to detach a listener:

```
EventBus.Unsubscribe<PlayerDiedEvent>(OnPlayerDied);
```

If the method was not subscribed, or was already removed, the call is ignored safely.

2.6 Publishing Events

Events are published using the `Publish<T>` method:

```
var evt = new PlayerDiedEvent("Alice");
EventBus.Publish(evt);
```

All listeners registered for `PlayerDiedEvent` will be notified immediately.

2.7 Implementation Summary

- Uses reflection-safe generic method calls.
- Event delegation is wrapped and tracked via `_delegateMap`.
- Subscribers are stored per event type.
- Supports any class implementing `IEvent`.

2.8 Example Event Type

```
public class PlayerDiedEvent : IEvent
{
    public string PlayerName { get; }

    public string Name => "PlayerDied";

    public PlayerDiedEvent(string playerName)
    {
        PlayerName = playerName;
    }
}
```

2.9 Threading Note

The `EventBus` is not thread-safe. All usage should occur from the main Unity thread unless explicitly modified for concurrency.

2.10 Use Cases

- Player health reaching zero → publish `PlayerDiedEvent`.
- Game state changes → publish `GameStateChangedEvent`.
- UI reacting to events without tight coupling.

3 Generic Singleton Base Class

3.1 Overview

The `Singleton<T>` class in `MLGWorks.Utils.Patterns` provides a reusable and type-safe implementation of the Singleton pattern for Unity components. It ensures that only one instance of a given `MonoBehaviour`-derived type exists in the scene at any time.

3.2 Purpose

The Singleton pattern is useful for manager-style components such as `GameManager`, `AudioManager`, or `UIController` that need to be globally accessible and guaranteed to have exactly one instance.

3.3 Usage

To create a singleton, derive from `Singleton<T>`:

```
public class GameManager : Singleton<GameManager>
{
    protected override void Awake()
    {
        base.Awake();
        // Additional setup here
    }

    protected override void OnDestroy()
    {
        base.OnDestroy();
        // Additional cleanup here
    }

    public void RestartGame()
    {
        // Game logic
    }
}
```

Access the singleton instance:

```
GameManager.Instance.RestartGame();
```

3.4 Key Features

- Ensures a single instance of `T` exists.
- Automatically finds the instance using Unity's `FindFirstObjectByType<T>()`.
- Destroys any duplicates found in the scene.
- Logs a warning when duplicates are destroyed.
- Offers `Awake()` and `OnDestroy()` override hooks for further customization.

3.5 Thread Safety

This implementation is not thread-safe; it relies on Unity's main thread operations.

3.6 Error Handling

If no instance is found in the scene when accessed via `Instance`, an `InvalidOperationException` is thrown:

```
throw new InvalidOperationException($"No {typeof(T)} found in scene.");
```

3.7 Limitations

- Cannot create the instance automatically—must be added to the scene manually.
- Not suitable for non-MonoBehaviour singletons.
- The base class does not enforce persistence across scenes (use `DontDestroyOnLoad` if needed).

4 State Machine

4.1 Overview

The `StateMachine` class in `MLGWorks.Utils.Patterns` provides a lightweight and extensible framework for implementing finite state machines. It allows you to define states via the `IState` interface and manage transitions and ticking logic.

4.2 Interfaces

```
public interface IState
{
    void OnEnter();
    void OnExit();
    void Tick();
}
```

Any class implementing `IState` can be used as a state in the state machine. The interface defines three methods:

- `OnEnter()`: Called when the state becomes active.
- `OnExit()`: Called when the state is deactivated.
- `Tick()`: Called once per frame while the state is active.

4.3 Usage

```
public class IdleState : IState
{
    public void OnEnter() => Debug.Log("Entering Idle");
    public void OnExit() => Debug.Log("Exiting Idle");
    public void Tick() => Debug.Log("Idling...");
}

public class PatrolState : IState
{
    public void OnEnter() => Debug.Log("Start Patrol");
    public void OnExit() => Debug.Log("Stop Patrol");
    public void Tick() => Debug.Log("Patrolling...");
}
```

Creating and using the state machine:

```
StateMachine fsm = new StateMachine();

IState idle = new IdleState();
IState patrol = new PatrolState();

fsm.ChangeState(idle); // Enters idle
fsm.Tick();            // Calls idle.Tick()

fsm.ChangeState(patrol); // Exits idle, enters patrol
fsm.Tick();            // Calls patrol.Tick()
```

4.4 Features

- Simple and intuitive API for switching states.
- Clean separation of logic per state.
- Ensures proper lifecycle management with `OnEnter` and `OnExit`.
- Decouples state logic from the state machine controller.

4.5 Best Practices

- Use this system for AI, gameplay logic, input handling, and other behavior switching tasks.
- Consider combining with a visual editor or scriptable objects for more advanced workflows.
- Extend with transition conditions, logging, or debugging utilities as needed.

5 Dependency Injection System

5.1 Overview

The `MLGWorks.Utils.DependencyInjection` system provides a lightweight and extensible approach to managing dependencies within Unity projects using a Service Locator pattern.

This system supports:

- Manual service registration for pure C# classes and select `MonoBehaviours`.
- Automatic dependency injection using attributes.
- Lifecycle initialization via an interface.
- Automatic registration of marked `MonoBehaviours`.

5.2 Registering Services

Pure C# Services

Pure C# services should be registered manually in a dedicated `ServiceBootstrapper` `MonoBehaviour` with a very early execution order:

```
// Example
ServiceLocator.Register<Logger>(new Logger());
ServiceLocator.Register<GameSettings>(new GameSettings());
```

`ServiceBootstrapper` is executed during the `Awake()` phase and is responsible for setting up non-`MonoBehaviour` dependencies early in the lifecycle.

MonoBehaviour Services

`MonoBehaviours` that are services and should be registered automatically must be marked with the `[DIService]` attribute:

```
[DIService]
public class MyService : MonoBehaviour { }
```

These services will be registered automatically during the `Awake()` phase of the `DIInitializer`, which runs after most other scripts.

5.3 Dependency Injection

To inject a service into a field, decorate it with the `[Inject]` attribute:

```
public class MyComponent : MonoBehaviour
{
    [Inject] private Logger _logger;

    private void Start()
    {
        _logger.Log("Injected successfully!");
    }
}
```

All `MonoBehaviours` in the scene (including inactive ones) are scanned and injected at runtime by the `DIInitializer`.

5.4 Lifecycle Initialization

If a `MonoBehaviour` needs to perform logic after injection is complete, it should implement the `IInitializable` interface:

```
public class MyComponent : MonoBehaviour, IInitializable
{
    [Inject] private GameSettings _settings;

    public void Initialize()
    {
        Debug.Log("Initialized with settings: " + _settings);
    }
}
```

`DIInitializer` will automatically call `Initialize()` after injecting dependencies.

5.5 Execution Order

- **ServiceBootstrapper:** Runs early (`DefaultExecutionOrder` = -1000). Registers core services.
- **DIInitializer:** Runs late (`DefaultExecutionOrder` = 10000). Injects dependencies and calls `Initialize()`.

5.6 Best Practices

- Register all non-`MonoBehaviour` services manually in `ServiceBootstrapper`.
- Use `[IService]` to mark `Monobehaviours` that should be registered automatically.
- Avoid registering the same type multiple times to prevent conflicts.
- Use `IInitializable` for post-injection setup logic.

6 Logger Utility

6.1 Overview

The `Logger` class in `MLGWorks.Utils.Logging` provides a flexible and centralized logging framework for Unity applications. It captures Unity logs, routes them into categorized log files, and supports features like file cleanup, multiple output targets, and per-level filtering.

6.2 LogLevel and Targets

Each log entry has a severity level:

- Debug
- Info
- Warning
- Error

Log levels are directed to specific files using 5-bit masks:

- Bit 0: Debug file
- Bit 1: Info file
- Bit 2: Warning file
- Bit 3: Error file
- Bit 4: Combined file (receives all)

Example:

```
public int debugTargets = (1 << 0) | (1 << 4); // debug + combined
```

6.3 Configuration Options

`LogLocationType` Determines the base path used for log storage:

- `PersistentDataPath`
- `DataPath`
- `Custom`

`relativePath` Folder path relative to base directory.

`customPath` Absolute path if `Custom` type is selected.

`fileExtension` Default: `.log`.

`maxLogFileCount` Limits how many timestamped log file sets are retained. Use `-1` to keep all.

`enableDebugLogging` Toggles whether `Debug()` logs are allowed.

6.4 Logging API

The logging methods enqueue messages for writing in a background-safe fashion:

- `Logger.Debug(string msg)`
- `Logger.Info(string msg)`
- `Logger.Warning(string msg)`
- `Logger.Error(string msg)`

```
Logger.Info("App started.");
```

All logs are timestamped and formatted like:

```
[14:23:01.123] [Info] App started.
```

6.5 Unity Integration

Unity messages (e.g., `Debug.Log`, `Debug.LogError`) are automatically captured via `Application.logMessageReceived` and redirected to the logger with a `[UNITY]` prefix.

6.6 Lifecycle

- Logging begins lazily when the first log is emitted.
- `Flush()` is called on every frame.
- Log files are closed on `OnDestroy()` and `OnApplicationQuit()`.

6.7 Log Cleanup

The method `CleanupOldLogFiles()` deletes old log groups based on timestamps:

- Files are grouped by prefix timestamp (e.g., `2025-07-20_12-30-00`).
- Groups exceeding `maxLogFileCount` are deleted oldest-first.

6.8 Example Usage

```
Logger.Debug("This is a debug log.");  
Logger.Info("App launched successfully.");  
Logger.Warning("Disk space running low.");  
Logger.Error("Unhandled exception occurred.");
```

6.9 Testing

`EmitTestLogs()` can be called to quickly generate log messages of all levels.

6.10 Editor Support

In the Unity Editor, `FlushAndShutdown()` can be used to immediately close files and clear writers without waiting for app shutdown.

6.11 Implementation Notes

- Uses a thread-safe `ConcurrentQueue` to stage logs.
- Writes are flushed in `Update()`.
- Log file creation uses timestamps to separate sessions.
- Each log level writes to target files based on its configured mask.